

一次 Blocking read () 的一生

把进程、文件、页缓存、I/O 与调度组合成一条可验证状态轨迹

408 · OS-I01

Canonical Problem

一个正在运行的进程调用 `read(fd, buf, n)`。若所需文件内容不能立即从内存状态满足，系统怎样从用户态进入内核，沿文件引用找到数据，把存储请求交给设备，让当前 `task` 安全等待，并在完成后重新获得执行机会，最终返回 “实际读取字节数 / EOF / error”？

Running Task → Syscall → fd/OFD/File Range → Cached/Resident? → I/O if needed → Block/Wake → Resum

箭头表示本次组合过程中的依赖与状态推进。Cache hit、EOF、error 等分支可以提前结束慢路径，但不能跳过相应合法性检查。

Integration Ownership

本册只 Owns 模块识别、组合次序、跨模块状态轨迹、分支与独立验证。

- OS-01/02 Own task、syscall 后的控制权、Blocked/Ready、scheduler/dispatcher；
- OS-06/07 Own fd/OFD/file object、current offset、file range 与 byte-to-block；
- OS-B03 Own process descriptor binding 与 open instance 的 handoff；
- OS-B04 Own file range、cached/resident page、VA mapping 与 I/O 之间的 handoff；
- OS-05 Own request、driver、device progress、completion；
- OS-B01 Own wait relation、block、wake、重新竞争 CPU 的接口；
- 若题目继续追问 interrupt/DMA 的硬件交接，再调用 X-B03。

本 Integration 不重新定义这些机制。

1 初始状态、目标与四条并行轨迹

题目一旦出现 `read()`，不要只画一条 “进程阻塞” 的线。至少同时追踪四条状态：

轨迹	初始问题	关键状态	成功时输出
Task / Control	谁正在执行？CPU 控制权在哪？	user/kernel、Running/Blocked/Ready、wait relation	task 最终在某次调度后继续，并按 syscall ABI 返回
File Reference	fd 指向谁？当前文件位置是什么？	fd binding、OFD/open instance、file object、offset	定位本次实际读取的 file range
Data / Residency	所需内容现在在哪里？	user buffer、cached file page、resident/dirty/I/O-in-flight	目标 bytes 已可从内存状态交付给本次 read
Device / Completion	若内存不足以满足，谁负责推进存储请求？	request queue、driver/device state、completion evidence	请求完成状态被软件确认，并可唤醒 waiter

成功条件不是 “磁盘读完了”，也不是 “进程被唤醒了”，而是：本次 read 的接口语义得到一个合法

返回结果，进程在以后真正获得 CPU 后把该结果返回用户态。

2 阶段一：从用户调用进入内核，但暂时还是同一个 task

进程在用户态执行 `read(fd, buf, n)`，通过系统调用入口把控制权交给内核。此时首先发生的是 `privilege/control transfer`，不应自动画成进程切换：

Mode/Privilege Entry $\Rightarrow$ Context Switch

内核先验证参数和当前调用语义所需的权限/对象条件。典型检查包括：`fd` 是否有效、打开实例是否允许读取、用户 `buffer` 是否可合法访问、请求长度是否合法。具体 ABI 与 `user-pointer` 校验机制由 OS-00/OS-01/02 的接口边界解释。

3 阶段二：沿 `fd` 找到“这次读的是文件的哪一段”

通过 OS-B03 的稳定关系：

Task $\xrightarrow{fd}$ Open Instance/OFD $\rightarrow$ File Object

内核得到本次操作依赖的打开实例状态，并确定逻辑读取起点。对普通顺序读，`current offset` 与“实际成功消费的 bytes”有关；如果题目使用 `pread()` 一类显式位置接口，则不能再假设共享 OFD offset 被推进。

于是文件系统把问题压成：

File Object + [`offset`, `offset + n`) 的实际可读部分

EOF、文件长度、设备/特殊文件语义都可能让本次调用在产生存储 I/O 之前就得到结果。

4 阶段三：先问内存状态能不能立即满足，而不是先画磁盘

OS-B04 要求先把文件内容身份与驻留状态分开：

1. **Cached/Resident Hit**：所需内容已有可用内存副本，本次 `read` 可以直接进入数据交付路径；通常不需要因“文件在磁盘上”而把 task 阻塞等待设备。
2. **Content Not Resident / Need More Data**：现有内存状态不足以满足本次读取，文件系统确定需要哪些后备存储数据，并向 OS-05 形成 I/O request。

第一个关键 Anti-Bridge

`read(file)` $\nRightarrow$ disk I/O

文件是持久对象，不代表每次访问都必须碰设备。`Page Cache / buffer hit` 可以让调用只走内存快路径；反过来，`cache miss` 也不自动等于“必须置换页面”，还要看是否有可用 `frame` 和具体分配状态。

5 阶段四：真正需要 I/O 时，把“请求推进”和“task 等待”拆成两条线

当数据必须由设备推进时：

1. OS-06/07 把 `file range` 翻译成下游可请求的数据位置/块关系；

2. OS-05 建立 request，交给 driver/controller/device；
3. 如果当前接口语义是 blocking 且结果尚不能返回，OS-B01 建立“当前 task 正在等待这个完成条件”的 wait relation，并让 task 失去 CPU 调度资格；
4. scheduler 从其他 Ready/Runnable 实体中选择下一位，必要时发生 context switch。

这里必须守住：

Submit I/O $\neq$ Block

异步接口可以提交后先返回；即使 blocking read，也可能因已有数据、partial result 或 error 而无需等待。是否 block 是调用者/接口状态，谁搬运数据是 I/O mechanism，两者是正交维度。

6 阶段五：设备完成先改变软件状态，再改变 task 的调度资格

设备请求在 CPU 之外继续推进。完成以后，系统必须先得到足够的 completion evidence，并把结果关联回正确 request：

Device Progress → Completion Detected → Request/Page State Updated → Wake Eligible Waiter

完成发现可能依赖 interrupt、polling 或混合方案；DMA 可能参与数据搬运，也可能不是题目所在抽象层。Integration 不把某一种硬件实现当定义。

最关键的状态边界：

I/O Completion → Ready/Runnable $\rightarrow$ Running

被唤醒的 task 只是重新进入 CPU 竞争集合。是否立即抢占当前运行者由调度策略、安全点与优先级等条件决定。

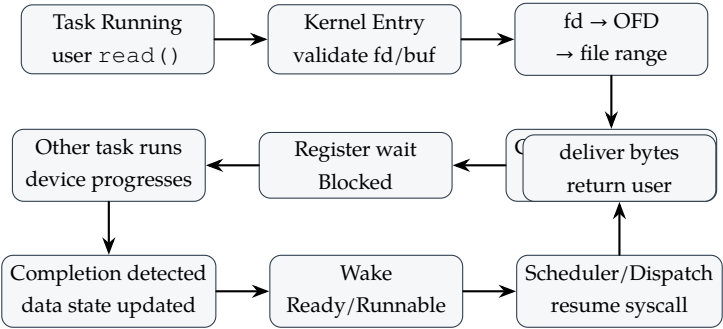
7 阶段六：重新运行以后，系统调用从“旧内核路径”继续

当调度器以后真正选中原 task：

1. 恢复该 task 的可恢复执行现场；
2. 重新检查本次 request / cached content 是否满足继续条件；
3. 把实际可交付数据推进到调用者 buffer（具体是否发生用户-内核复制取决于路径与接口）；
4. 更新本次操作拥有的文件位置/返回状态；
5. 通过 syscall return 恢复到用户态，返回实际字节数、0（例如 EOF）或 error。

注意 read(fd, buf, n) 的成功返回值不承诺一定等于 n。题目若涉及 pipe、terminal、socket、signal interruption 或 partial I/O，应按对应对象接口重新判断；本册的 canonical mother example 是普通 file-backed blocking read。

8 完整母轨迹：Cache Miss 的普通文件读取



这张图里只有两条地方真正代表 CPU execution context change: task block 后换给别人, 以及以后 scheduler 重新选择原 task。Syscall entry/return 是 privilege transition; device completion 是事件; wake 是资格变化。把四者混成“上下文切换”会直接破坏计算题计数。

9 高价值分支：同一句 read，轨迹为什么不同

条件变化	轨迹变化	不应错误推出
Page Cache hit	跳过设备 request 与 I/O wait，直接交付可用数据	“read 一定阻塞”
到达 EOF	可直接返回 0 或已读取的 partial bytes（按接口状态）	“必须读满 <i>n</i> 才返回”
无空闲 frame	VM 可能先需要 reclaim/replacement，再形成/完成 page-in	“cache miss 就等于 replacement”
I/O 完成时 CPU 正跑别的 task	waiter 先 Ready，是否抢占再由 scheduler 决定	“interrupt 一来原 task 立即继续”
同一 OFD 被 fork/dup 共享	成功读取消费的 current offset 可能影响另一引用者	“不同 fd 数字就有独立 offset”
使用 pread()	显式位置读取，不按普通 current offset 推进	“任何 read-like API 都共享 OFD offset”
使用 mmap()	文件内容通过 VA mapping + fault/load 访问，不再以每次 read syscall 为主线	“mmap 只是更快的 read”
Direct/async I/O	cache/copy/blocking 关系可能改变	“DMA/async/nonblocking 是同义词”

10 失败定位：找第一个不能满足的前置条件

现象	首个候选 Owner	第一验证动作
立即返回 bad fd / permission error	OS-06/07 + OS-B03	fd binding、打开模式、file object 是否合法
地址/用户 buffer 错误	OS-01/02 + OS-04 Use	user pointer 的映射/权限是否允许当前 copy
文件存在但读到 EOF	OS-06/07	current offset 与 file size/range
长时间阻塞	OS-05 + OS-B01	request 是否提交、等待哪个 completion、是否已 wake
设备完成但进程未运行	OS-01/02	task 是否 Ready、scheduler 是否已选择它
读到内容但 offset 推演错	OS-B03	是否共享同一 OFD、实际返回多少 bytes

11 独立验证与停止条件

完成一道 blocking read 综合题后，固定做六次检查：

1. **Object**：fd、OFD、file object、file range、cached page、I/O request、task 是否被当成不同对象？
2. **Control**：syscall entry、block/context switch、interrupt/completion、wake、dispatch 是否分开？
3. **Queue/Relation**：task 真正因什么条件进入哪个 wait relation，什么事件解除它？
4. **Data**：数据此刻到底在 persistent storage、cache/resident memory 还是 user buffer？
5. **Accounting**：offset 只按实际消费的 bytes 与具体接口契约变化，是否误把请求长度 n 当实际完成量？
6. **Stop**：如果题目没问 DMA/MMU/文件系统持久化细节，就在对应 Bridge 接口停止，不复制 Topic 内部机制。

Compression

遇到“阻塞读”不要背一条固定故事，按四问复原：

读哪一个 file range？→ 内存能否立即满足？→ 若不能，谁推进 request、task 等什么？→ completion 后为什么？

能回答这四问，OS-01/02、OS-05、OS-06/07 与 B01/B03/B04 就已经正确组合起来。